If you're newer to Rails, you might be wondering how changes are captured in the db/structure.sql file. When a developer makes a schema modification, behind the scenes Ruby on Rails when configured for PostgreSQL uses pg_dump to dump the table structure to db/structure.sql.

If you've used a schema modification management tool like Flyway or Liquibase, Active Record Migrations are going to be somewhat similar. In Ruby on Rails, Active Record is built-in and is the way developers evolve the schema.

Each modification is called a "Migration", which is an overloaded term. In Rails a Migration is a versioned Ruby file in the db/migrate directory. Rails Migrations use an Active Record API that maps Ruby methods to SQL DDL changes.

Within the Ruby source code file, schema modifications can be written in SQL as well. Do Rails Migration files only contain structural changes, aka Data Definition Language (DDL)? No, there aren't any limitations on them, so developers can perform Data Modification Language (DML) statements in the form of inserting, updating, or deleting data. However, it's considered a good practice to use Migrations only for DDL and use Rake tasks or other mechanisms for DML.

Does the db/structure.sql contain only structural data about the schema? Although nearly all of the content of the file is purely structural, there is one exception. At the bottom of the file there is "data". The versioned Migration files each have a unique number. The number is inserted into the developer's database when the Migration runs.

How is this tracked then? When the structure file is dumped, it's dumped based on the developer's database, so any migrations that have been applied there are dumped as INSERT statements. The schema migrations are dumped as INSERT statements so that the database structure can be replicated on another database by running bin/rails db:schema:load.

What this task does is load the content from the structure file and then insert all the same migration versions into the destination database. That way if another migration comes in after that one with a version number that doesn't exist, the database knows which versions have already been applied and will apply only the new ones.

That was a quick refresher on how Active Record Migrations manage the PostgreSQL database structure, how developers make incremental schema modifications, and how those modifications are tracked.